Practical:8.1: Services

Aim-To study and implement an Android Service that performs a background task (such as a counter or timer) without direct user interaction.

Problem Statement – Develop an Android application that implements a Started Service which runs a background counter (or timer) and logs the count every second using Logcat.

The service should: Start when a button is clicked.

Continue running even when the activity is minimized.

Stop when a stop button is pressed.

Display output in Logcat or Toast messages for verification.

STEP 1: Create Project-Open Android Studio

Click New Project

Select: Empty Views Activity

Click Next

Configure Project

Field	**Value**
Name	StartedServiceCounter
Package Name	com.example.startedservicecounter
Language	**Java**
Minimum SDK	API 21 (or default)

Click Finish

Wait for Gradle sync.

STEP 2: Create Service Class

Right Click on:

app > java > com.example.startedservicecounter

New → Java Class
Name it: CounterService

STEP 3: Write CounterService.java

Replace everything with this:

```
package com.example.startedservicecounter;

import android.app.Service;

import android.content.Intent;

import android.os.Handler;
```

```
import android.os.IBinder;
import android.util.Log;
import android.widget.Toast;
public class CounterService extends Service {
    private Handler handler;
    private Runnable runnable;
    private int counter = 0;
    private boolean isRunning = false;
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        if (isRunning) {
            Log.d("CounterService", "Service already running");
            return START_STICKY;
        }
        isRunning = true;
        handler = new Handler();
        runnable = new Runnable() {
            @Override
            public void run() {
                counter++;
                Log.d("CounterService", "Counter: " + counter);
                handler.postDelayed(this, 1000);
            }
        };
        handler.post(runnable);
        Toast.makeText(this, "Service Started", Toast.LENGTH_SHORT).show();
        return START_STICKY;
    }
    @Override
    public void onDestroy() {
        super.onDestroy();
```

```
        if (handler != null && runnable != null) {
            handler.removeCallbacks(runnable);
        }
        isRunning = false;
        counter = 0;
        Toast.makeText(this, "Service Stopped", Toast.LENGTH_SHORT).show();
        Log.d("CounterService", "Service Destroyed");
    }
    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
}
```

STEP 4: Register Service in AndroidManifest.xml

Open:

app > manifests > AndroidManifest.xml

Inside <application> tag add this:

```
<service
    android:name=".CounterService"
    android:enabled="true"
    android:exported="false"/>
```

Very important step
Without this → app crashes.

STEP 5: Design Layout

Open:res > layout > activity_main.xml

Replace everything with:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="20dp">
    <Button
        android:id="@+id/btnStart"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Start Service"/>
    <Button
        android:id="@+id/btnStop"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Stop Service"
        android:layout_marginTop="20dp"/>
</LinearLayout>
```

STEP 6: Write MainActivity.java

Open:MainActivity.java

Replace everything with:

```
package com.example.startedservicecounter;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    Button btnStart, btnStop;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
```

```
        btnStart = findViewById(R.id.btnStart);
        btnStop = findViewById(R.id.btnStop);
        btnStart.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Intent intent = new Intent(MainActivity.this, CounterService.class);
                startService(intent);
            }
        });
        btnStop.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Intent intent = new Intent(MainActivity.this, CounterService.class);
                stopService(intent);
            }
        });
    }
}
```

STEP 7: Run the App

What Should Happen?

Click Start Service
Counter starts in Logcat
Toast appears

Minimize app
Service keeps running (check Logcat)

Click Stop Service
Counter stops
"Service Stopped" Toast

How to See Logcat

Bottom → **Logcat**

Filter by:CounterService

You'll see: Counter: 1,Counter: 2,Counter: 3

Practical:8.2

Aim :To study and implement Android Notification to alert users about application events.

Problem Statement :Develop an Android application that generates a Notification when a button is clicked.

The notification should: Display a title and message.

Appear in the notification panel.

Open the application when clicked.

Project Name: Notification

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:gravity="center"
   android:orientation="vertical">
   <Button
      android:id="@+id/btnNotify"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Generate Notification" />
</LinearLayout>
```

MainActivity.java

```
package com.example.notification;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.content.Intent;
import android.os.Build;
import android.os.Bundle;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.NotificationCompat;
public class MainActivity extends AppCompatActivity {
   private static final String CHANNEL_ID = "notification_channel";
```

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    Button btnNotify = findViewById(R.id.btnNotify);
    // Create Notification Channel (Required for Android 8+)
    createNotificationChannel();
    btnNotify.setOnClickListener(view -> {
        // Intent to open app when notification clicked
        Intent intent = new Intent(MainActivity.this, MainActivity.class);
        intent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP);
        PendingIntent pendingIntent = PendingIntent.getActivity(
                MainActivity.this,
                0,
                intent,
                PendingIntent.FLAG_IMMUTABLE
        );
        // Build Notification
        NotificationCompat.Builder builder =
                new NotificationCompat.Builder(MainActivity.this, CHANNEL_ID)
                        .setSmallIcon(android.R.drawable.ic_dialog_info)
                        .setContentTitle("Notification Alert")
                        .setContentText("Button Clicked! This is your notification.")
                        .setPriority(NotificationCompat.PRIORITY_DEFAULT)
                        .setContentIntent(pendingIntent)
                        .setAutoCancel(true);
        // Show Notification
        NotificationManager notificationManager =
                (NotificationManager) getSystemService(NOTIFICATION_SERVICE);
        notificationManager.notify(1, builder.build());
    });
}
// Method to create Notification Channel
private void createNotificationChannel() {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        CharSequence name = "Notification Channel";
        String description = "Channel for app notifications";
        int importance = NotificationManager.IMPORTANCE_DEFAULT;
        NotificationChannel channel =
                new NotificationChannel(CHANNEL_ID, name, importance);
        channel.setDescription(description);
        NotificationManager manager =
                getSystemService(NotificationManager.class);
        if (manager != null) {
            manager.createNotificationChannel(channel);
        }
    }
```

```
    }
}
```

AndroidManifest.xml

Make sure this line exists:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Notification">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Final Output When you:

Run app

Click **Generate Notification**

Pull notification panel

You will see: Notification Alert
Button Clicked! This is your notification.

Click it App opens
Notification disappears automatically

Practical Completion Statement (For Record)
The Android application was successfully developed to generate a notification on button click. The notification displays title and message in the notification panel and opens the application when clicked.

Practical:09

Aim: To study database programming using SQLite in Android.

Problem Statement: Develop an Android application using SQLite database to create a table and perform insert, update, delete and display operations on the stored records.

What is SQLite?

SQLite is a lightweight database built inside Android.

No installation needed
No internet required
Works offline
Data stored inside app

What We Will Build

A simple **Student Record App**

Table: student

Columns: id (Primary Key), name, marks

Operations: Insert student Update student Delete student Display all students

STEP 1 — Create Project

1. Open Android Studio
2. Click New Project
3. Select Empty Views Activity
4. Click Next

Project Details

Field	**Value**
Name	SQLiteDemo
Language	Java
Min SDK	21

Click **Finish**

Wait for Gradle to build.

STEP 2 — How SQLite Works in Android

Android gives a helper class called:

SQLiteOpenHelper

We must create our own class extending this.

This class:Creates database

Creates table

Handles version changes

STEP 3 — Create DatabaseHelper Class

Right click:

app > java > yourpackage

New → Java Class
Name it: DatabaseHelper

DatabaseHelper.java

```
package com.example.sqlitedemo;

import android.content.ContentValues;

import android.content.Context;

import android.database.Cursor;

import android.database.sqlite.SQLiteDatabase;

import android.database.sqlite.SQLiteOpenHelper;

public class DatabaseHelper extends SQLiteOpenHelper {

   public static final String DATABASE_NAME = "Student.db";

   public static final String TABLE_NAME = "student";

   public static final String COL_1 = "ID";

   public static final String COL_2 = "NAME";

   public static final String COL_3 = "MARKS";

   public DatabaseHelper(Context context) {

      super(context, DATABASE_NAME, null, 1);

   }

   @Override

   public void onCreate(SQLiteDatabase db) {

      db.execSQL("CREATE TABLE " + TABLE_NAME +

            " (ID INTEGER PRIMARY KEY AUTOINCREMENT, NAME TEXT, MARKS INTEGER)");

   }

   @Override

   public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {

      db.execSQL("DROP TABLE IF EXISTS " + TABLE_NAME);
```

```
        onCreate(db);
    }
    public boolean insertData(String name, String marks) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put(COL_2, name);
        contentValues.put(COL_3, marks);
        long result = db.insert(TABLE_NAME, null, contentValues);
        return result != -1;
    }
    public Cursor getAllData() {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.rawQuery("SELECT * FROM " + TABLE_NAME, null);
    }
    public boolean updateData(String id, String name, String marks) {
        SQLiteDatabase db = this.getWritableDatabase();
        ContentValues contentValues = new ContentValues();
        contentValues.put(COL_1, id);
        contentValues.put(COL_2, name);
        contentValues.put(COL_3, marks);
        db.update(TABLE_NAME, contentValues, "ID = ?", new String[]{id});
        return true;
    }
    public Integer deleteData(String id) {
        SQLiteDatabase db = this.getWritableDatabase();
        return db.delete(TABLE_NAME, "ID = ?", new String[]{id});
    }
}
```

STEP 4 — Design Layout (activity_main.xml)

Replace with:

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="20dp">
        <EditText
            android:id="@+id/editId"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Enter ID" />
        <EditText
            android:id="@+id/editName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Enter Name" />
        <EditText
            android:id="@+id/editMarks"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Enter Marks" />
        <Button
            android:id="@+id/btnAdd"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Insert" />
```

```
        <Button
            android:id="@+id/btnView"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="View All" />
        <Button
            android:id="@+id/btnUpdate"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Update" />
        <Button
            android:id="@+id/btnDelete"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Delete" />
    </LinearLayout>
</ScrollView>
```

STEP 5 — MainActivity.java

Replace code with:

```
package com.example.sqlitedemo;
import androidx.appcompat.app.AppCompatActivity;
import android.database.Cursor;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
public class MainActivity extends AppCompatActivity {
    DatabaseHelper myDb;
    EditText editName, editMarks, editId;
```

```
Button btnAddData, btnViewAll, btnUpdate, btnDelete;
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    myDb = new DatabaseHelper(this);
    editId = findViewById(R.id.editId);
    editName = findViewById(R.id.editName);
    editMarks = findViewById(R.id.editMarks);
    btnAddData = findViewById(R.id.btnAdd);
    btnViewAll = findViewById(R.id.btnView);
    btnUpdate = findViewById(R.id.btnUpdate);
    btnDelete = findViewById(R.id.btnDelete);
    AddData();
    viewAll();
    UpdateData();
    DeleteData();
}
public void AddData() {
    btnAddData.setOnClickListener(v -> {
        boolean isInserted = myDb.insertData(
                editName.getText().toString(),
                editMarks.getText().toString()
        );
        if (isInserted)
            Toast.makeText(MainActivity.this, "Data Inserted", Toast.LENGTH_LONG).show();
        else
            Toast.makeText(MainActivity.this, "Data Not Inserted", Toast.LENGTH_LONG).show();
    });
}
```

```
public void viewAll() {
    btnViewAll.setOnClickListener(v -> {
        Cursor res = myDb.getAllData();
        if (res.getCount() == 0) {
            Toast.makeText(MainActivity.this, "No Data Found", Toast.LENGTH_LONG).show();
            return;
        }
        StringBuffer buffer = new StringBuffer();
        while (res.moveToNext()) {
            buffer.append("ID :" + res.getString(0) + "\n");
            buffer.append("Name :" + res.getString(1) + "\n");
            buffer.append("Marks :" + res.getString(2) + "\n\n");
        }
        Toast.makeText(MainActivity.this, buffer.toString(), Toast.LENGTH_LONG).show();
    });
}
public void UpdateData() {
    btnUpdate.setOnClickListener(v -> {
        boolean isUpdate = myDb.updateData(
                editId.getText().toString(),
                editName.getText().toString(),
                editMarks.getText().toString()
        );
        if (isUpdate)
            Toast.makeText(MainActivity.this, "Data Updated", Toast.LENGTH_LONG).show();
        else
            Toast.makeText(MainActivity.this, "Data Not Updated", Toast.LENGTH_LONG).show();
    });
}
public void DeleteData() {
    btnDelete.setOnClickListener(v -> {
```

```
        Integer deletedRows = myDb.deleteData(editId.getText().toString());
        if (deletedRows > 0)
            Toast.makeText(MainActivity.this, "Data Deleted", Toast.LENGTH_LONG).show();
        else
            Toast.makeText(MainActivity.this, "Data Not Deleted", Toast.LENGTH_LONG).show();
    });
  }
}
```

Run The App

Test:

Insert → Enter name & marks

View → Shows all records

Update → Enter ID + new values

Delete → Enter ID

Concept	**Meaning**
SQLiteOpenHelper	Manages database
onCreate()	Creates table
insert()	Adds data
rawQuery()	Reads data
update()	Modifies data
delete()	Removes data

Practical10

Runtime Camera Permission in Android (Java)

Aim: To study and implement security features and runtime permission handling in Android applications.

Problem Statement: Develop an Android application that requests Camera permission at runtime. When the user clicks a button, the application should: Check whether the Camera permission is granted. If not granted, request the permission from the user. Display a message indicating whether the permission is granted or denied. The application should handle permission approval and denial properly.

From Android 6.0 (Marshmallow) onwards, permissions like Camera, Location, etc. must be requested at runtime, not only in the Manifest.

There are 3 main steps in runtime permission handling:

Declare permission in AndroidManifest.xml

Check permission using ContextCompat.checkSelfPermission()

Request permission using ActivityCompat.requestPermissions()

Handle result inside onRequestPermissionsResult()

We will implement exactly this.

Project Setup

Project Name: CameraPermissionApp

Language: Java
Template: Empty Views Activity
Minimum SDK: API 23 or above

1. AndroidManifest.xml

Add Camera permission inside <manifest> but outside <application>.

```
<uses-permission android:name="android.permission.CAMERA"/>
```

2. activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout

    xmlns:android="http://schemas.android.com/apk/res/android"

    android:layout_width="match_parent"

    android:layout_height="match_parent"

    android:gravity="center"

    android:orientation="vertical">
```

```
    <Button
        android:id="@+id/btnCamera"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Request Camera Permission"/>
</LinearLayout>
```

3. MainActivity.java

```
package com.example.camerapermissionapp;
import android.Manifest;
import android.content.pm.PackageManager;
import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
public class MainActivity extends AppCompatActivity {
    private static final int CAMERA_REQUEST_CODE = 100;
    Button btnCamera;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        btnCamera = findViewById(R.id.btnCamera);
        btnCamera.setOnClickListener(v -> checkCameraPermission());
    }
    private void checkCameraPermission() {
        // Step 1: Check if permission already granted
        if (ContextCompat.checkSelfPermission(this,
                Manifest.permission.CAMERA) == PackageManager.PERMISSION_GRANTED) {
```

```
            Toast.makeText(this,
                    "Camera Permission Already Granted",
                    Toast.LENGTH_SHORT).show();
        } else {
            // Step 2: Request permission
            ActivityCompat.requestPermissions(this,
                    new String[]{Manifest.permission.CAMERA},
                    CAMERA_REQUEST_CODE);
        }
    }
    // Step 3: Handle user response
    @Override
    public void onRequestPermissionsResult(int requestCode,
                                           @NonNull String[] permissions,
                                           @NonNull int[] grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);
        if (requestCode == CAMERA_REQUEST_CODE) {
            if (grantResults.length > 0
                    && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
                Toast.makeText(this,
                        "Camera Permission Granted",
                        Toast.LENGTH_SHORT).show();
            } else {
                Toast.makeText(this,
                        "Camera Permission Denied",
                        Toast.LENGTH_SHORT).show();
            }
        }
    }
}
```

What happens when button is clicked?

checkCameraPermission() is called.

App checks:

If permission already granted → show message.

If not granted → ask user for permission.

1. System shows permission dialog.
2. User selects Allow or Deny.
3. onRequestPermissionsResult() receives result.
4. Based on result → show proper message.

How This Implements Security

Runtime permission:

- Prevents apps from accessing camera without user knowledge.
- Gives user full control.
- Improves privacy.
- Prevents misuse of sensitive features.